

# ntiny Core

## Microarchitecture Reference

A bottom-up, module-by-module description of the ntiny RISC-V core, its memory subsystem, and its behaviour under variable memory latency.

RV32IMACZicsr\_Zifencei + Sv32 + PMP · 5-stage in-order

generated from `docs/microarch/`

## Contents

|          |                                                      |          |
|----------|------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                  | <b>2</b> |
| 1.1      | The pipeline, in one breath . . . . .                | 2        |
| 1.2      | The one idea that explains the hard bugs . . . . .   | 2        |
| 1.3      | How to read the diagrams . . . . .                   | 2        |
| <b>2</b> | <b>Instruction cache (icache)</b>                    | <b>3</b> |
| 2.1      | Purpose . . . . .                                    | 3        |
| 2.2      | Geometry . . . . .                                   | 3        |
| 2.3      | Ports . . . . .                                      | 4        |
| 2.4      | How a request is served . . . . .                    | 4        |
| 2.5      | Why it is latency-tolerant . . . . .                 | 4        |
| 2.6      | Fill FSM . . . . .                                   | 5        |
| <b>3</b> | <b>Load/Store unit (core2avl)</b>                    | <b>6</b> |
| 3.1      | Purpose . . . . .                                    | 6        |
| 3.2      | Ports . . . . .                                      | 6        |
| 3.3      | How it works . . . . .                               | 7        |
| 3.4      | The IWB control latch and variable latency . . . . . | 7        |

## 1 Introduction

This document describes the **ntiny** core one module at a time, bottom-up: the memory leaves first, then the caches, the bus, the load/store and atomic units, the core control logic, and finally how they compose into **core\_top** and the SoC. Each module gets a block diagram, a port summary, a plain-language walk-through, and—where it matters—a note on how it behaves when memory does *not* answer in a single cycle.

### 1.1 The pipeline, in one breath

ntiny is a classic in-order pipeline. Instructions flow through five stages:

IF (fetch) → ID (decode/align) → IE (execute) → IMEM (memory) → IWB (writeback).

A fetch front-end (PC logic, instruction cache, a fetch buffer, and a compressed aligner) feeds a decoder; the execute stage resolves branches and runs the ALU; the memory stage issues data accesses through a shared data bus; writeback commits results and CSR side effects. Forwarding paths and a central **hazard\_unit** keep the pipeline correct without software-visible stalls.

### 1.2 The one idea that explains the hard bugs

**Latency note.** The original pipeline was *statically scheduled* around a memory that always answers in exactly one cycle. With single-cycle memory, “the load’s data is on the bus exactly when the consumer needs it,” so no dynamic interlock is required—correctness falls out of the fixed schedule. The moment memory latency becomes *variable* (a real cache miss, **+RAM\_RANDOM\_DELAY**, a future AXI/DRAM slave), that invariant breaks everywhere a memory result is produced or consumed: forwarding, branch/JALR resolution, the writeback commit gate, the fetch in-flight tracking, the bus response routing, and the cache itself. Each such site needs an explicit “data-not-ready-yet” handshake. This is why “just more wait time” is deceptively invasive—it is not one stall point, it is the removal of a global timing assumption. Throughout this document, **Latency notes** flag exactly where that assumption lived and what replaced it.

### 1.3 How to read the diagrams

Every diagram uses one consistent visual vocabulary so shapes always mean the same thing:

| shape / colour         | meaning                                               |
|------------------------|-------------------------------------------------------|
| yellow rectangle       | combinational logic (decode, mux net, ALU op)         |
| blue rectangle         | register / flip-flop (pipeline reg, latch, counter)   |
| green stadium          | a state machine (state register + transitions)        |
| grey hexagon           | multiplexer / select                                  |
| purple cylinder        | memory array (SRAM, cache tag/data arrays)            |
| white box              | an instantiated sub-module                            |
| pale boxes<br>(in/out) | module ports, grouped left (inputs) / right (outputs) |
| solid arrow            | data / datapath signal                                |
| dotted arrow           | control / handshake / stall / freeze                  |

Diagrams are deliberately *sparse*: node labels are short signal or block names, and all the detail lives in the prose of each section. The diagram sources are Mermaid (`docs/microarch/diagrams/*.mmd`); rebuild the figures and this PDF with `python3 docs/microarch/build.py`.

## 2 Instruction cache (icache)

Source: `design/memory/src/icache.sv`

### 2.1 Purpose

The `icache` sits between the fetch front-end and instruction-side RAM (port A). It answers instruction fetches from a small on-chip array when it can (a *hit*), and fetches a missing word from RAM (a *miss*) otherwise. It is a **single-outstanding stalling slave**: it can hold the fetch unit off (`cpu_ready=0`) while it services a miss, and it tolerates RAM that takes an arbitrary number of cycles to answer.

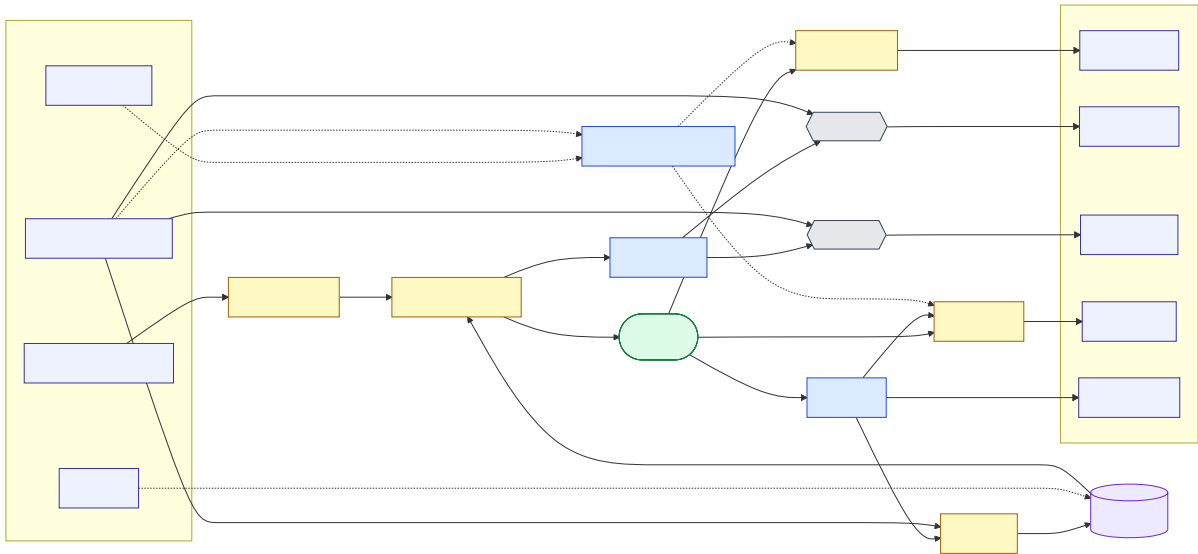


Figure 1: `icache` datapath. Left: CPU/fetch and RAM inputs. Right: CPU and RAM outputs. The FSM, the captured fill registers, and the `mem_outstanding` tracker are what make it latency-tolerant.

### 2.2 Geometry

4KB, 4-way set-associative, 32-byte lines (8 words per line), 32 sets. An address splits as `tag[31:10]`, `index[9:5]`, `word_off[4:2]`, byte offset `[1:0]` (ignored—fetches are word-aligned). Each line carries a `line_valid` bit *and* per-word `word_valid` bits, so a single-word fill installs only the word that was returned; a hit needs both bits set. Replacement is round-robin per set (`repl_ptr`).

## 2.3 Ports

| port                       | dir | meaning                                     |
|----------------------------|-----|---------------------------------------------|
| cpu_req_i / cpu_addr_i     | in  | fetch request + word address                |
| cpu_rdata_o                | out | fetch instruction word                      |
| cpu_rvalid_o               | out | response valid (one pulse per accepted req) |
| cpu_ready_o                | out | 1 = can accept a new request this cycle     |
| flush_i                    | in  | FENCE.I: invalidate all lines               |
| mem_req_o / mem_addr_o     | out | RAM read request to port A                  |
| mem_rdata_i / mem_rvalid_i | in  | RAM read response                           |
| mem_ready_i                | in  | RAM accepts a request this cycle            |

## 2.4 How a request is served

**Address decode and hit test** are combinational on `cpu_addr_i`: the four ways are checked in parallel for (`line_valid` AND tag match AND `word_valid`); `hit` is their OR and `hit_way_id` selects the matching way.

**On a hit**, the cache registers a one-cycle response: `resp_valid_q` pulses next cycle and `resp_data_q` holds the selected word. `cpu_ready` stays high, so hits accept back-to-back at one fetch per cycle—the same behaviour fetch always saw.

**On a miss**, the FSM (§2.6) leaves `S_IDLE` for `S_FILL`, drops `cpu_ready` to 0 (the fetch unit now *holds* its address and request stable), captures the request into `fill_tag/index/word/ addr_q` with `fill_live_q`=1, and drives `mem_req` to RAM. When the RAM read returns (`mem_rvalid`), the word is installed into the array and forwarded to the CPU as `cpu_rvalid` with `mem_rdata`; the FSM returns to `S_IDLE`.

**Install** (`install_tag_match`) is recomputed against the *current* array state at write time, so back-to-back fills of consecutive words in the same line land in the way the previous fill just allocated instead of picking a fresh way (this avoids a two-ways-same-tag corruption seen earlier in Linux init).

**FENCE.I** (`flush_i`) invalidates every line in one cycle and blocks new accepts; a fill already in flight is abandoned but its RAM read is still drained (below).

## 2.5 Why it is latency-tolerant

**Latency note.** The previous icache was *transparent*: `cpu_ready`=`mem_ready`, `mem_addr`=`cpu_addr` combinationally, and the response was derived from the *live* `cpu_addr`. That silently assumes RAM returns data exactly one cycle after the request. Under variable latency the fetch unit has already advanced `cpu_addr` to the next word by the time the data arrives, so the transparent cache returned the *wrong-offset* word (e.g. it answered a fetch of `0x..04` with the word at `0x..08`), which double-executed branch targets and corrupted `beq/bne/cj` results. The fix is the structure in Fig. 1: `cpu_ready` drops during a fill so the master holds; the fill address is *captured* and used for the RAM request, the install slot, and the response; and `mem_outstanding_q` keeps the RAM port strictly single-outstanding (so an abandoned FENCE.I fill's late response is swallowed before a new fill can issue). Under always-ready RAM `mem_outstanding` clears in one cycle, so a miss simply costs one stall cycle—baseline behaviour is unchanged.

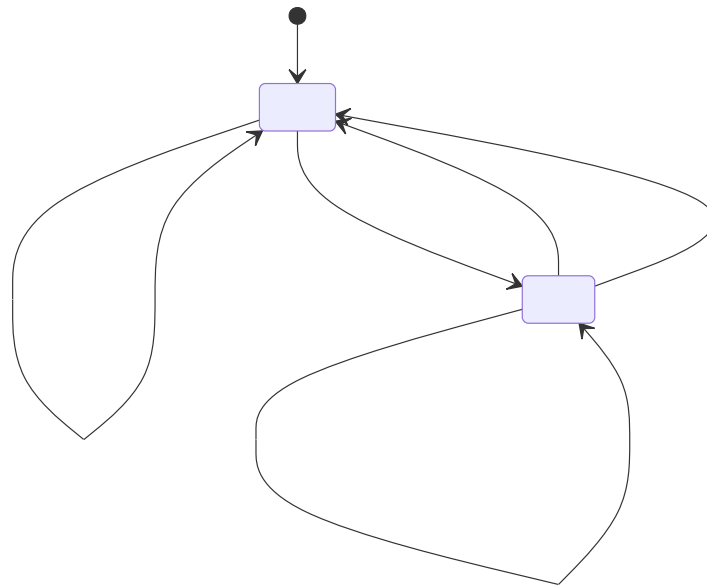


Figure 2: icache fill FSM. A hit is served from S\_IDLE in one cycle; a miss enters S\_FILL, holds the master off, issues the captured RAM read, and waits for `mem_rvalid`.

## 2.6 Fill FSM

| from   | to     | when / action                                                                                       |
|--------|--------|-----------------------------------------------------------------------------------------------------|
| S_IDLE | S_IDLE | accept_hit: register <code>resp_valid_q</code> for next cycle                                       |
| S_IDLE | S_FILL | accept_miss: capture <code>fill_*_q</code> , set <code>fill_live</code>                             |
| S_FILL | S_FILL | drive <code>mem_req</code> until <code>mem_ready</code> accepts, then await <code>mem_rvalid</code> |
| S_FILL | S_IDLE | <code>fill_live &amp; mem_rvalid</code> : install word, pulse <code>cpu_rvalid</code>               |
| S_FILL | S_IDLE | <code>flush_i</code> abandons the fill; RAM read drained, no <code>cpu_rvalid</code> owed           |

### 3 Load/Store unit (core2avl)

Source: `design/core/avalon_master/src/core2avl.sv`

#### 3.1 Purpose

`core2avl` is the CPU's load/store unit (LSU). It turns a decoded memory operation (a load or store of a byte/half/word at some address) into bus transactions, and formats returning read data (sign- or zero-extended, shifted to the right byte lanes). A naturally-aligned access is one bus beat; a half/word that straddles a 32-bit word boundary is split into two aligned beats by a small FSM.

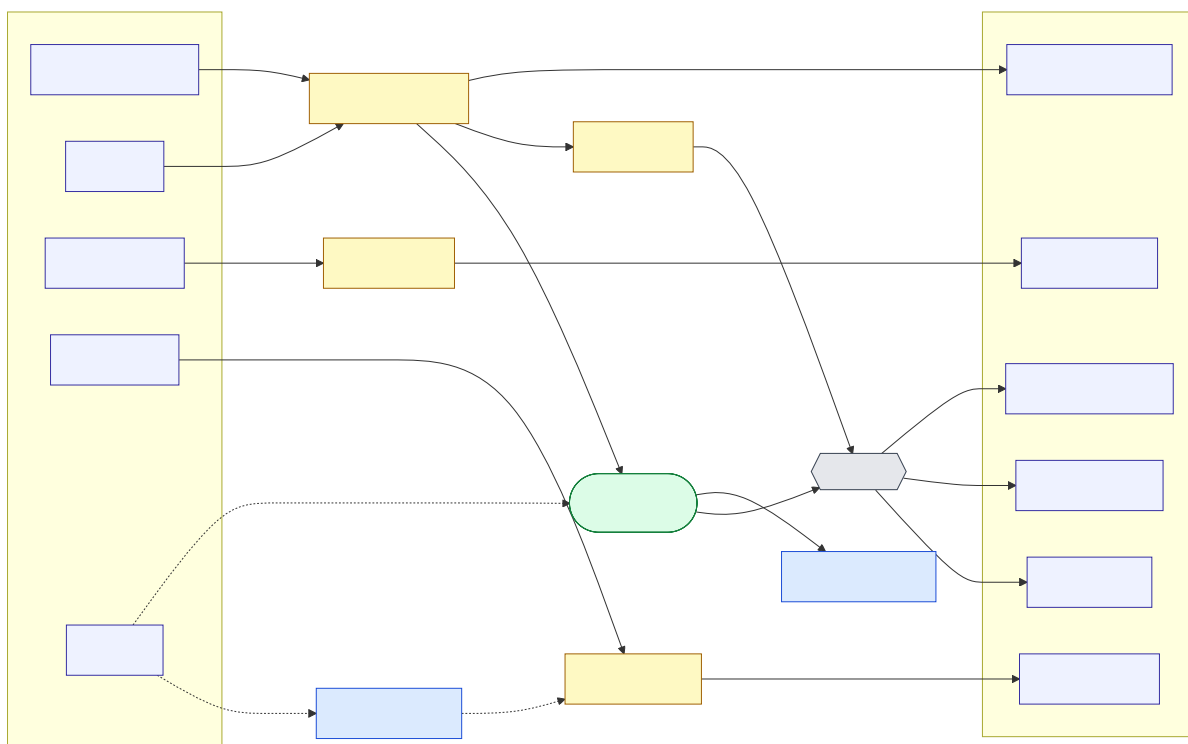


Figure 3: `core2avl` datapath. Address/op enter on the left; bus request signals and the formatted read result leave on the right. The IWB ctrl latch (blue) holds the formatting controls across a multi-cycle load.

#### 3.2 Ports

| port                                                  | dir | meaning                                                        |
|-------------------------------------------------------|-----|----------------------------------------------------------------|
| <code>mem_op / load_store_width / mem_unsigned</code> | in  | op kind, size, signedness                                      |
| <code>addr_i</code>                                   | in  | access address (from the ALU)                                  |
| <code>data2write_i</code>                             | in  | store data                                                     |
| <code>readdata_i</code>                               | in  | read data from the bus                                         |
| <code>stall_i</code>                                  | in  | freeze the IWB control latch + FSM (= <code>iwb_stall</code> ) |
| <code>address_o / byteenable_o / writedata_o</code>   | out | bus request fields                                             |
| <code>read_o / write_o</code>                         | out | bus request strobes                                            |
| <code>data2read_o</code>                              | out | formatted load result (to IWB)                                 |
| <code>misalign_stall_o</code>                         | out | 1 during the first beat of a crossing access                   |

### 3.3 How it works

**Misalign detect** computes, combinatorially from the width and `addr[1:0]`, whether the access crosses a word boundary. If not, the access is a single aligned beat: `address_o=addr_i`, `byteenable_o` is the byte-lane mask for that size/offset, and `writedata_o` is the store data shifted into the correct lanes.

**Crossing accesses** use the IDLE → SECOND FSM. The first beat reads/writes the low part at the aligned address; the operands (`addr_q`, `wdata_q`, `width_q`, `op_q`) and the first read word are latched; the next cycle issues the second beat at the next word, and the two halves are recombined on the read path. `misalign_stall_o` holds the pipeline for the extra beat.

**Read formatting** (`read_extract`) selects the right bytes from `readdata_i` using a small *IWB control latch*—`be_iwb`, `mode_iwb`, `mem_unsigned_iwb`, `byt_iwb`—that was captured when the access was issued, then applies sign/zero extension to produce `data2read_o`.

### 3.4 The IWB control latch and variable latency

**Latency note.** `read_extract` formats *live* `readdata_i` using the *registered* controls in the IWB ctrl latch. Those controls are clocked by `stall_i`. Originally `stall_i` was tied to 0, so the latch updated every cycle. With single-cycle memory that is fine: the read data arrives the one cycle the controls are still valid. Under variable latency the load's data arrives several cycles later—by which point the *next* instruction (often a non-load) has overwritten `be_iwb` to “no byte lanes,” so the returning word formats to 0. The fix is to drive `stall_i=iwb_stall`: the latch (and the FSM) freeze while the load waits, so `be_iwb` survives until `readdata_i` is valid. Because `iwb_stall` is permanently 0 under always-ready RAM, baseline behaviour is unchanged.